



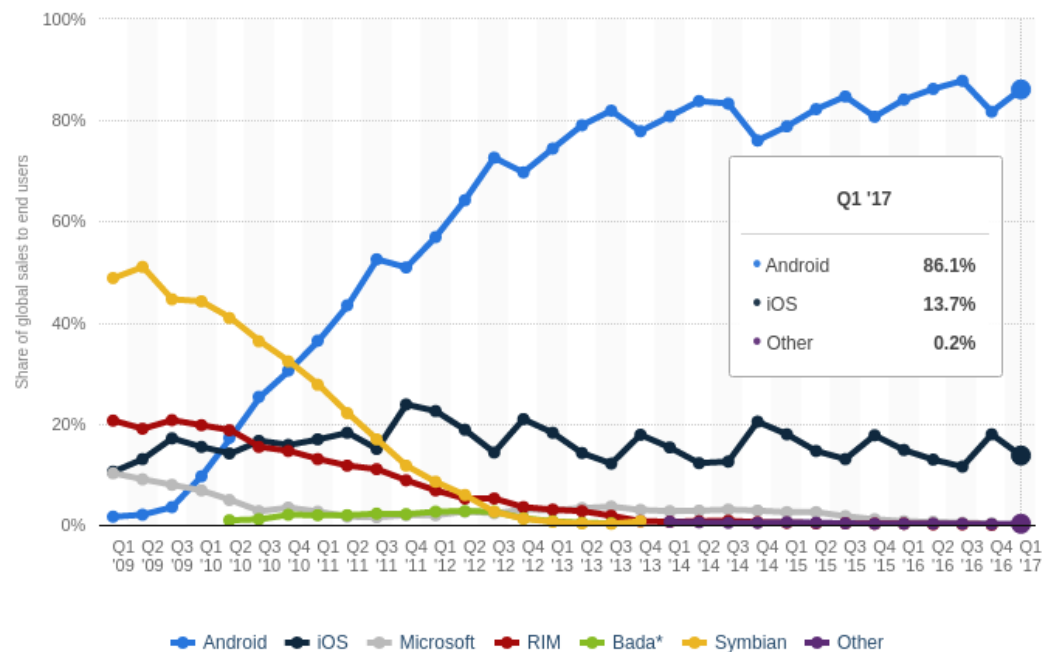
Table of Contents

- 1.1) Introduction of Mobile OSes: Android, iOS, Ubuntu, Touch, Blackberry, Tizen, Firefox OS, Symbian, Windows Phone
- 1.2) Build and Structures of Mobile Oses
- 1.3) Introduction to development environment (Native v/s HTML5)
- 1.4) Introduction to Android
 - 1.4.1 API levels/versions of Android
 - 1.4.2. Pros and Cons of Android
 - 1.4.3. Comparison of Android with other Mobile Oses
- 1.5) Introduction to Android VM and Runtime (Dalvik and ART)
- 1.6) Installation and configuration of Android SDKs and Eclipse IDE
 - 1.6.1. Their integration using ADT Plugin
 - 1.6.2. Running an emulator
- 1.7) Using ADB command line interface

Mobile Operating system – Market share

1.1) Introduction of Mobile OSes: Android, iOS, Ubuntu, Touch, Blackberry, Tizen, Firefox OS, Symbian, Windows Phone

Global market share held by the leading smartphone operating systems in sales to end users from 1st quarter 2009 to 1st quarter 2017



ABOUT THIS STATISTIC

This statistic shows the global market share held by the leading smartphone operating systems, in terms of sales to end users, from 2009 to 2017. In the first quarter of 2017, 86.1 percent of all smartphones sold to end users were phones with the Android operating system.

[Show more](#)

SPECIAL FUNCTIONS

Download as ...

Graphic (PNG)

Excel (XLS)

PowerPoint (PPT)

PDF

Options

Settings

Print

Research Alerts

Table of Contents

1.1) Introduction of Mobile OSes: Android, iOS, Ubuntu, Touch, Blackberry, Tizen, Firefox OS, Symbian, Windows Phone

Mobile Operating System is an Operating System specifically designed for mobile devices. Like. PDAs, Smartphones, Tablets or other kind of hand held with touchscreen facilities.

Laptops are also known as mobile but the operating system made for typical laptop was designed for large desktop machine hence those are not called mobile operating system. But newer operating system could come with hybrid in nature so we can use it on computer or the mobile devices.

Typically most of the common connectivity feature of computer system is embedded into mobile devices and will carry some distinct feature as well, the modern mobile system will have touchscreen, cellular, Blue tooth, WiFi, GPS, GPS navigation, Camera, Video Camera, Speech recognition, voice recording, biometric authentication and many...

There are various mobile operating system available in the market. Some of them are open source and some of them are proprietary based.

Currently there are few operating system which are very popular in the market. Most successful mobile operating system in the planet today is Android which has almost 80% market share worldwide as of 2015 and it is open source. Followed by iOS which is developed by Apple inc.

There are other operating system as well like windows phone and blackberry.

In summary, Mobile OS is a software that is required to provide a platform or an environment for a user to execute their required programs on a mobile devices in a convenient and efficient manner.

Table of Contents



Table of Contents



Table of Contents



- Note that Android based on a Linux kernel not a Linux OS
- Security, Memory management, Process management, Network stack and Driver model
- Acts as an abstraction layer between the hardware and the rest of the software stack

Table of Contents



Run in system background

Using C/C++ Language

4 types of Libraries

- a) Bionic Libc, system C libraries
- b) Function Libraries, supporting multimedia, web browser, SQLite...
- c) Native Servers
- d) Hardware Abstraction Libraries

System C library, the standard C system library, tuned for embedded Linux-based devices

Media Libraries, support playback and recording of many popular audio and video formats, as well as image files,

including MPEG4, H.264, MP3, AAC, AMR, JPG, and PNG **Surface Manager**, manages access to the display subsystem and seamlessly composites 2D and 3D graphic layers from multiple applications **WebKit**, a modern web browser engine which powers both the Android browser and an embeddable web view **SGL**, the underlying 2D graphics engine **3D libraries**, an implementation based on OpenGL ES 1.0 APIs **FreeType**, bitmap and vector font rendering **SQLite**, a powerful and lightweight relational database engine



The core of Android platform

→ Dalvik Virtual Machine

Register-based

Executes files in the Dalvik Executable (.dex) format

→ Java core Libraries

- a) Provides most of the functionality of the Java programming language.
- b) The functions of Java core libraries rely on the Dalvik VM and the underlying Linux kernel
- c) Multiple Dalvik VMs may run at the same time
- d) Every Android application runs in its own process, with its own instance of the Dalvik virtual machine
- e) The "dx" tool in Android SDK can transform compiled JAVA class into the .dex format

Table of Contents

There are quite a few factors that determine how to select the best technology for your mobile app. Each method of developing a mobile app has its own set of advantages and disadvantages. In this blog post, we will talk about the main difference between native, hybrid and HTML5 mobile app development. In addition to this, I will explain some pros and cons of each method. So, let's star



1. Native Apps

Apps that are platform specific, that is – Android or iOS are called native apps. Native apps have a specific development environment and they are built within that only. Native apps provide a simplified environment for development functions such as debugging and project management.

Advantages

1. These apps can be downloaded from the App store and can be used instantly by the end user. The developer and the user, both find managing a Native app very simple.
2. Native apps integrate with the mobile device features like camera, location, address book etc. These apps also integrate with other apps to benefit from additional features.
3. There is a huge community out there to help Native App developers build better apps. Native apps are relatively more secure.
4. Native apps are developed in the respective environment and benefit from the upgrades in the respective platforms

Native App Disadvantages

1. One major drawback of a native app is that it is restricted to a particular platform.
 2. Native apps require more investment as it requires separate resources to design, develop and maintain apps for all the respective platforms.
 3. Managing native apps is difficult, as any changes that need to be done, have to go through a repeated process for all three platforms.
-

Native App Disadvantages

1. One major drawback of a native app is that it is restricted to a particular platform.
 2. Native apps require more investment as it requires separate resources to design, develop and maintain apps for all the respective platforms.
 3. Managing native apps is difficult, as any changes that need to be done, have to go through a repeated process for all three platforms.
-

2. HTML5 Mobile Apps

HTML5 apps can work on different mobile platforms. They use a combination of programming languages like CSS, Javascript, and HTML5. Technically, an HTML5 mobile app is a web page that works as an app. Since the source of this kind of an application is not the platform, the app becomes platform neutral.

The challenge here is to develop apps that fit a variety of screen sizes. The overall user experience of native apps, however, is considered to be superior to HTML5 Apps. On the other hand, HTML5 apps save time and effort, as one has to write these apps once and then they can be used for multiple platforms. From an overall user's perspective, native apps can be made more user-friendly as they fit the requirements of the platform for which they are being built.

Advantages

1. Reduced costs and time required to build these apps
2. Business consistency across platforms
3. These apps are easier to maintain as compared to native apps

Disadvantages

1. HTML 5 apps cannot benefit from the upgrade features of the respective platforms
2. Creating large and heavy apps (like gaming) is difficult to manage with HTML5
3. HTML5 Apps cannot be used when offline
4. These apps run on a browser and lack the desired quality from a UX perspective

Table of Contents

3. Hybrid Mobile Apps

Hybrid mobile apps development use a combination of Native and HTML5 development methodologies to develop applications that have the advantages of both. A native container is used to develop hybrid apps while programming languages used are CSS Javascript and HTML5. These apps are available on different platforms and can be downloaded from the respective app stores.

Since users demand the same apps on different platforms, businesses are increasingly switching over to this methodology.

Hybrid apps are faster to develop and also provide a great user experience. They definitely bring the benefits of both Native and HTML5 methodologies.

Advantages

1. These apps do not require an approval and can be updated while the users are using them
2. The application is consistent across platforms
3. Common web analytics tools can be used to analyze the traffic on hybrid apps
4. Reduced costs and time

Disadvantages

1. Hybrid apps cannot benefit from the upgrade features of the respective platforms
2. Creating large and heavy apps (like gaming) is difficult to manage with the Hybrid model
3. The UX quality of Hybrid apps does not match that of Native



By: Arun Kumar Maskey

API level/version of Android

As the Android platform evolves and new Android versions are released, each Android version is assigned a unique integer identifier, called the API Level. Because users install apps on older as well as the most recent versions of Android, real-world Android apps must be designed to work with multiple Android API levels. And each of the android version is named on the basis of dessert name.

Nougat	7.1	API level 25
Nougat	7.0	API level 24
Marshmallow	6.0	API level 23
Lollipop	5.1	API level 22
Lollipop	5.0	API level 21
KitKat	4.4 - 4.4.4	API level 19
Jelly Bean	4.3.x	API level 18
Jelly Bean	4.2.x	API level 17
Jelly Bean	4.1.x	API level 16
Ice Cream Sandwich	4.0.3 - 4.0.4	API level 15, NDK 8
Ice Cream Sandwich	4.0.1 - 4.0.2	API level 14, NDK 7
Honeycomb	3.2.x	API level 13

API level/version of Android

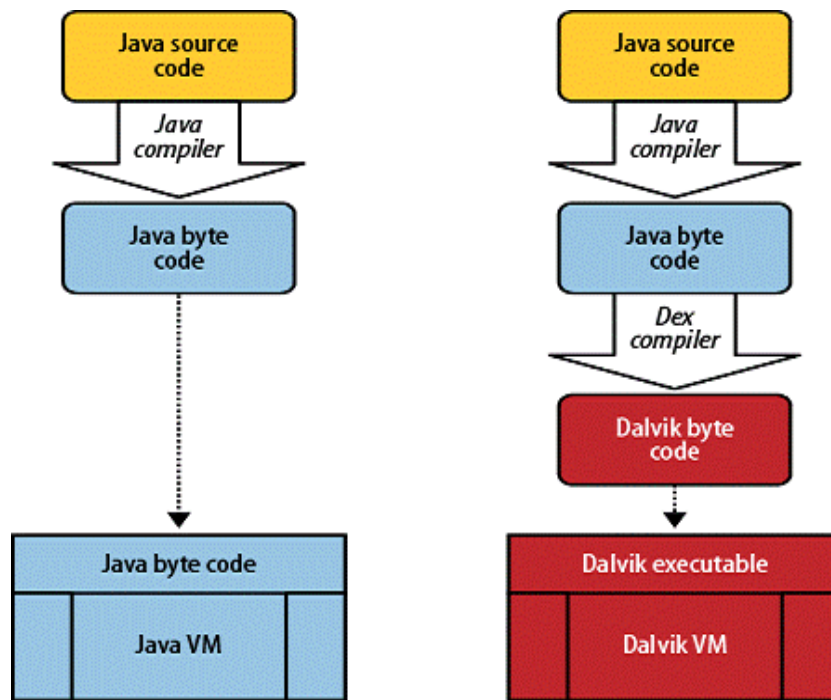
Honeycomb	3.1	API level 12, NDK 6
Honeycomb	3.0	API level 11
Gingerbread	2.3.3 - 2.3.7	API level 10
Gingerbread	2.3 - 2.3.2	API level 9, NDK 5
Froyo	2.2.x	API level 8, NDK 4
Eclair	2.1	API level 7, NDK 3
Eclair	2.0.1	API level 6
Eclair	2.0	API level 5
Donut	1.6	API level 4, NDK 2
Cupcake	1.5	API level 3, NDK 1
(no code name)	1.1	API level 2
(no code name)	1.0	API level 1

As this table indicates, new Android versions are released frequently – sometimes several releases per year. As a result, the universe of Android devices that might run your app includes of a wide variety of older and newer Android versions. How can you guarantee that your app will run consistently and reliably on so many different versions of Android? Android's API levels can help you manage this problem.

Android API Level

Each Android device supports exactly one API level – this API level is guaranteed to be unique per Android platform version. The API level exactly identifies the version of the libraries that your app can call into; it identifies the combination of manifest elements, permissions, etc. that you code against as a developer. Android's system of API levels helps Android determine whether an application is compatible with an Android system image prior to installing the application on a device. When an application is built, it contains the following API level information: The target API level of Android that the app is built to run on. The minimum API level of Android that is required to run the app. These settings are used to ensure that the functionality needed to run the app correctly is available on the Android device at installation time. If not, the app is blocked from running on that device. For example, if the API level of an Android device is lower than the minimum API level that you specify for your app, the Android device will prevent the user from installing your app

Understanding Dalvik VM and ART



Dalvik is a process virtual machine (VM) in Google's Android operating system that executes applications written for Android. This makes Dalvik an integral part of the Android software stack (in Android versions 4.4 "KitKat" and earlier) that is typically used on mobile devices such as mobile phones and tablet computers, as well as more recently on devices such as smart TVs and wearables.

Programs are commonly written in Java and compiled to bytecode for the Java virtual machine, which is then translated to Dalvik bytecode and stored in **.dex** (Dalvik EXecutable) and **.odex** (Optimized Dalvik EXecutable) files; related terms odex and de-odex are associated with respective bytecode conversions. The compact Dalvik Executable format is designed for systems that are constrained in terms of memory and processor speed. It was originally written by Dan Bornstein

Understanding ART

Android Runtime (ART) is an application runtime environment used by the Android operating system. ART replaces Dalvik, which is the process virtual machine originally used by Android, and performs transformation of the application's bytecode into native instructions that are later executed by the device's runtime environment.

Android 2.2 "Froyo" brought trace-based just-in-time (JIT) compilation into Dalvik, optimizing the execution of applications by continually profiling applications each time they run and dynamically compiling frequently executed short segments of their bytecode into native machine code. While Dalvik interprets the rest of application's bytecode, native execution of those short bytecode segments, called "traces", provides significant performance improvements.

Unlike Dalvik, ART introduces the use of ahead-of-time (AOT) compilation by compiling entire applications into native machine code upon their installation. By eliminating Dalvik's interpretation and trace-based JIT compilation, ART improves the overall execution efficiency and reduces power consumption, which results in improved battery autonomy on mobile devices. At the same time, ART brings faster execution of applications, improved memory allocation and garbage collection (GC) mechanisms, new applications debugging features, and more accurate high-level profiling of applications.